WarpDial: A Case for Reconfigurability and Explainability in Object Storage

Anonymous Author(s)
Affiliation withheld for review

Abstract—Object storage latency is a well-known problem, and why it is so large is rarely explained to the consumers of the storage layer. Managed services make this abstraction possible by taking responsibility for safety, availability, reliability, and speed; giving customers direct control over low-level storage decisions is therefore dangerous, since a poor configuration can compromise the guarantees the service exists to provide. We argue explainability and reconfigurability belong in the abstraction itself: a managed service should explain why a configuration costs what it does. This preliminary work explores one narrow part of that goal by asking whether per-bucket placement policies provide meaningful benefits compared to a single global policy, and what operational overhead comes with making placement configurable. Meeting this can require a fixed [4], parametric [5], or content-dependent [1] tunability layer; we explore the content-dependent case here: a reconfigurable granularity knob that governs how an object is striped (sharded) across storage, capable of Fusion-style [1] stripe alignment. This is a standalone, simulated prototype, evaluated separately on two structurally different workloads, Parquet column chunks and IVF vector-index partitions. A dense sweep shows that the right granularity depends on the workload. As granularity changes, byte amplification and round trips move in opposite directions. How much this matters also depends strongly on query selectivity. A single global default can leave real performance on the table. The flexibility has a cost too. This policy's location-map metadata grows with granularity. A workload-blind default never pays that cost, but it never gets the benefit either.

I. Introduction

Object storage latency is a widely recognized problem. AI training, databases, and agentic frameworks have all spent years building bespoke client-side workarounds for it [2]. Object storage let people build freely in the first place because it hid infrastructure complexity behind a simple interface. There was no hardware to manage, just PUT and GET. What is still unexamined is why the latency behind that simple interface is so large. Developers on a foundational layer, such as a database or a filesystem, have traditionally understood that layer well enough to reason about downstream tradeoffs. Object storage does not offer this by design. That opacity matters more, not less, now that agentic systems also build on top of it unattended, not only humans. They need to reason about its behavior themselves.

That opacity is the product managed services sell. They take responsibility for safety, availability, reliability, and speed so customers do not have to manage the infrastructure underneath. This is why handing customers direct control over low-level decisions is dangerous on its own. A poor configuration can break the guarantees the service exists to provide. More systems software is now built on top of managed services rather than raw infrastructure. We believe explainability and reconfigurability belong in the abstraction itself. A service should explain why a configuration costs what it does. It should not just expose a knob and leave a customer to guess. A bucket's SLA should be a derived output of its policy, not an asserted number.

Part of that opacity is structural. Object stores typically apply one placement and erasure-coding policy across an entire pool. That policy is chosen once and stays workload-blind. The same code runs whether a bucket holds cold backups or latency-sensitive analytics tables. WarpDrive [2] responds to this. Systems software keeps rebuilding the same primitives on top of managed storage. WarpDrive argues these primitives belong inside the storage layer instead, tunable and explainable, rather than leaving each side to compensate for the other's opacity.

Custom placement policies differ most in metadata cost. Computed placement derives location purely from the object key, bucket policy, and node set, as in CRUSH-style hashing [4]. Its cost is $O(\text{buckets})$. Parametric placement stores a small, fixed-size record per object, such as a seed, an epoch, or a layout version [5]. Its cost is $O(\text{objects})$, tiny and constant. Content-dependent placement derives the location map from the object's own data, through bin-packing by chunk size or content-defined chunking. Its cost is $O(\text{objects} \times \text{units per object})$. This cost grows with the data's own structure.

Workload-specific designs are one example of a content-dependent policy, not the foundation this poster argues for. Fusion's [1] file-format-aware erasure coding is one such design. It aligns stripe boundaries to a Parquet column chunk. This nearly eliminates fragmentation, at a cost of $\sim$1.2% storage overhead. We generalize its bin-packing rule into a knob. That knob is what we prototype and evaluate below.

II. Idea & Design

A content-dependent policy, concretely. We generalize Fusion's Algorithm 1 into a format-agnostic bin packer. The original algorithm seeds each stripe with its largest item, then packs the rest greedily into the least-occupied bin. Our packer takes only an arbitrary $(unit_id, size)$ list and a stripe width k. It has no format-specific logic. The

computable unit is a column chunk for Parquet, or an IVF partition for a vector index. Aligning stripes to this unit is what makes the policy content-dependent. The location map depends on the object's own data, not just its key.

The knob. A bucket's policy fixes two things. The first is the granularity, a batch size b. This controls how many consecutive finest-grained units are co-located per storage unit. b=1 never splits a unit. b=N collapses the bucket to one unit. The second is the EC parameters (k, m). The boundary list itself, where the units fall inside a given object, is still supplied by the client at PUT time (x-warpd-computable-units: [[offset,len],...]). That part is content-dependent by definition, so only the client's own data can determine it.

Metrics in tension. We measure three things. The first is storage overhead versus optimal. The second is two read-side costs that pull in opposite directions as granularity varies: byte amplification, and round trips, the number of distinct bins a query touches. Round trips are our proxy for network-RTT-bound latency. The third is metadata cost, one location-map entry per storage unit. Section III prices this from Fusion's own reported ∼8 bytes per chunk, replicated k+1-way. Finer granularity wastes fewer bytes per bin. But it touches more bins, and creates more metadata. Coarser granularity does the reverse.

III. Preliminary Results

Setup. We use two workloads. The first is a synthetic TPC-H-lineitem-shaped Parquet table: 16 columns, 10 row groups, 500,000 rows, 160 units. The second is 500,000 synthetic 768-dim vectors split into an IVF index. Partition count follows FAISS's own guidance, nlist = $4\sqrt{N}$ [3], giving 2,828 partitions. Partition sizes follow a Dirichlet-multinomial skew, since real IVF clusters are unbalanced. Queries sample units uniformly by selectivity. We use RS(9,6) erasure coding. Each point is a median over N=25 trials, with [25th, 75th] percentile bands. We sweep three selectivity regimes, 1%, 10%, and 50%, and a dense range of granularities per workload: 8 settings for Parquet, 9 for the vector index.

Reproduction check. FAC at Parquet column-chunk granularity reaches 1.16% storage overhead versus optimal. Fusion's own paper reports ∼1.2% on their real 10 GB file. The two numbers are the same order of magnitude. This validates our generalized packer before we trust anything built on top of it.

Finding 1: the right granularity is workload-dependent, evidence a custom bucket policy helps. Consider batch size 16. For Parquet, this is exactly one row group. For the vector index, it is an arbitrary batch of 16 partitions, structurally unrelated to anything in that format. At this same batch size, Parquet's storage overhead is 79.99%. The vector index's is 2.69%. That is a ∼29.8× gap between two structurally different workloads. Neither cost curve predicts the other's (Fig. 2). A bucket operator who

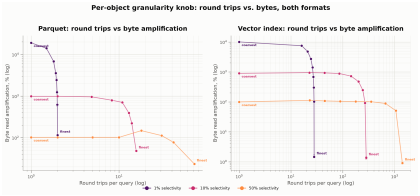


Fig. 1. Round trips vs. byte read amplification, both formats, three selectivity regimes. Each colored curve is the granularity-knob frontier (finest → coarsest) at one fixed query selectivity; selectivity itself is workload context, not a tuned knob, so curves are never connected across colors.

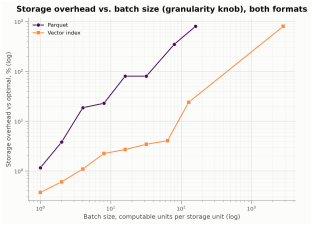


Fig. 2. Storage overhead vs. batch size (the granularity knob), both formats. Overhead is a packing property, independent of query selectivity.

hard-codes one granularity across both workloads loses on whichever one it was not tuned for.

Finding 2: round trips and bytes trade off, and the size of that tradeoff changes sharply with query selectivity. At 50% selectivity on Parquet, the finest granularity costs 71.9 round trips at 23.4% byte overhead. The coarsest granularity costs a single round trip at 101.9% byte overhead. That is a ∼4.4× gap. At 1% selectivity, the same two extremes are 2.0 round trips at 114.8% bytes versus 1.0 round trip at 19,040% bytes. That gap is ∼166×. The byte cost of the wrong choice grows sharply as selectivity drops. Which point is preferable depends on whether the workload is latency-bound or bandwidth-bound. A fixed default cannot express that distinction. A reconfigurable one can.

Finding 3: the operational cost of exposing this policy is concrete and small. We apply Fusion's own reported location-map cost, ∼8 bytes per chunk, replicated k+1-way. Parquet's metadata ranges from 12,800 bytes at the finest granularity, 160 units, down to 80 bytes at the coarsest. The vector index's metadata ranges from 226,240 bytes, 2,828 units, down to the same 80 bytes. Computed and parametric placement do not have this property. This policy's metadata grows directly with how finely the knob is tuned. That is the price of the flexibility Finding 1 shows is worth having.

IV. Conclusion & Future Work

A managed service only earns the right to hide complexity if it can still explain the choices it makes on a customer's behalf. We tested one representative content-dependent policy. Letting a bucket choose its own placement granularity helps. The right setting is workload-dependent. Getting it wrong costs sharply more as selec-

tivity drops, and metadata cost rises alongside it. This is a small step toward a storage layer where reconfigurability never costs safety. Every guarantee should be derived and explained, not asserted. The other policy classes, SLAs as policy outputs, and a solver over declared intents remain as future work.

Acknowledgment

Portions used AI-agent assistance, per HiPC's disclosure policy.

References

[1] Y. Lu, S. Raina, A. Cidon, and M. J. Freedman, "Fusion: An Analytics Object Store Optimized for Query Pushdown," in Proc. ASPLOS, 2025.

[2] Anonymous, "WarpDrive: A Composable Storage Substrate for Disaggregated Data Systems," CDMS Workshop, VLDB 2026.

[3] FAISS wiki, "Guidelines to Choose an Index."

[4] S. A. Weil et al., "CRUSH: Controlled, Scalable, Decentralized Placement of Replicated Data," in Proc. SC, 2006.

[5] L. Wang et al., "MapX: Controlled Data Migration in the Expansion of Decentralized Object-Based Storage Systems," in Proc. USENIX FAST, 2020.